Lab 11

PL/pgSQL Named Blocks: Functions and Procedures

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data Generation **5**

3 Introduction to Named Blocks **7**
3.1 User-Defined Functions 7
3.2 Stored Procedures 7

4 Parameter Modes **7**

5 Defining the Subprograms **8**

6 Execution Inside Anonymous Blocks **8**
6.1 Calling a Function 9
6.2 Calling a Procedure 9

7 Lab Tasks **10**

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

1. Connect to Your Database:

Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Verify the Connection:

Verify that you are connected to the correct database with the following command:

```
\conninfo
```

Expected output:

```
Database: cse4308_12345, User: student_12345
```

3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

Load the SQL file that sets up the tables and inserts initial data.

1. Load the SQL File:

Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_11_setup.sql
```

This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Check Tables:

Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L11_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

2 Database Schema & Data Generation

Instructions: Save this code as `lab_11_setup.sql` and run it using `\i lab_11_setup.sql` in your `psql` terminal. This script generates an e-commerce schema.

```sql
-- 1. CLEANUP
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS restock_logs CASCADE;
DROP TABLE IF EXISTS products CASCADE;
DROP TABLE IF EXISTS customers CASCADE;

-- 2. CREATE TABLES
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    region VARCHAR(50),
    loyalty_tier VARCHAR(20) -- 'Bronze', 'Silver', 'Gold', 'Platinum'
);

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    category VARCHAR(50),
    base_price NUMERIC(10, 2),
    stock_quantity INT
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date DATE,
    status VARCHAR(20), -- 'PENDING', 'PROCESSED', 'PARTIAL', 'CANCELLED'
    total_value NUMERIC(12, 2) DEFAULT 0.00
);

CREATE TABLE order_items (
    item_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id INT REFERENCES products(product_id),
    quantity INT,
    unit_price NUMERIC(10, 2)
);

CREATE TABLE restock_logs (
    log_id SERIAL PRIMARY KEY,
    product_id INT REFERENCES products(product_id),
    restock_date DATE,
    added_quantity INT
);

-- 3. DATA GENERATION
-- Insert 1000 Customers
INSERT INTO customers (name, region, loyalty_tier)
SELECT
    'Customer_' || id,
    (ARRAY['North', 'South', 'East', 'West'])[floor(random() * 4 + 1)],
    (ARRAY['Bronze', 'Silver', 'Gold', 'Platinum'])[floor(random() * 4 + 1)]
```

```sql
FROM generate_series(1, 1000) AS id;

-- Insert 200 Products
INSERT INTO products (name, category, base_price, stock_quantity)
SELECT
    'Product_' || id,
    (ARRAY['Electronics', 'Clothing', 'Groceries', 'Tools', 'Books'])[floor(random() *
     5 + 1)],
    ROUND((random() * 500 + 10)::numeric, 2),
    floor(random() * 100 + 10)::INT
FROM generate_series(1, 200) AS id;

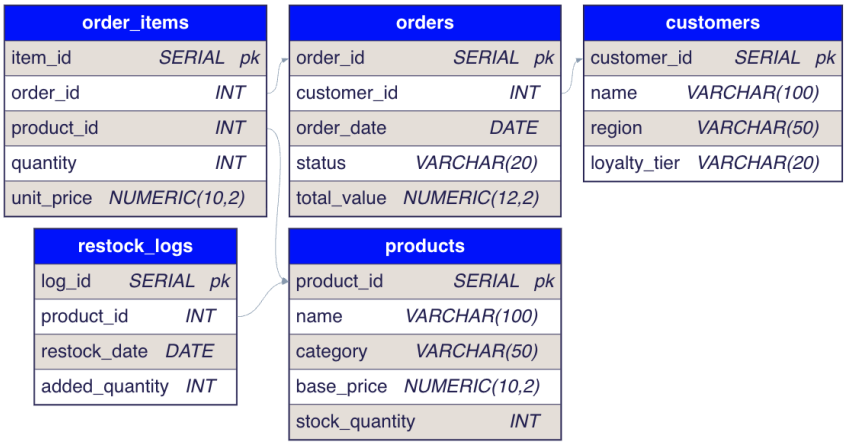
-- Insert 3000 Orders (Statuses randomized)
INSERT INTO orders (customer_id, order_date, status)
SELECT
    floor(random() * 1000 + 1)::INT,
    CURRENT_DATE - (floor(random() * 365)::INT),
    (ARRAY['PENDING', 'PROCESSED', 'PROCESSED', 'PARTIAL', 'CANCELLED'])[floor(random
    () * 5 + 1)]
FROM generate_series(1, 3000);

-- Insert 9000 Order Items (Averaging 3 items per order)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    floor(random() * 3000 + 1)::INT,
    floor(random() * 200 + 1)::INT,
    floor(random() * 5 + 1)::INT,
    0.00
FROM generate_series(1, 9000) ON CONFLICT DO NOTHING;

-- Fix the unit_price in order_items based on products table
UPDATE order_items oi
SET unit_price = p.base_price
FROM products p
WHERE oi.product_id = p.product_id;

-- Initial aggregation to set order total_values
UPDATE orders o
SET total_value = sub.total
FROM (
    SELECT order_id, COALESCE(SUM(quantity * unit_price), 0) as total
    FROM order_items
    GROUP BY order_id
) sub
WHERE o.order_id = sub.order_id;
```

Listing 1: lab_11_setup.sql



3 Introduction to Named Blocks

Anonymous blocks (`DO $$`) evaluate procedural logic for single execution but are not persistently stored in the database schema. Persistent named blocks, referred to as subprograms, encapsulate procedural logic for reuse. In PostgreSQL, subprograms are classified into two primary constructs: **User-Defined Functions** and **Stored Procedures**.

3.1 User-Defined Functions

A named PL/pgSQL block that evaluates to a specific return type. Functions are invoked within expressions (e.g., a `SELECT` statement) and structurally require a `RETURN` or `RETURN QUERY` statement. They execute entirely within the transaction scope of the invoking query; they cannot execute Transaction Control Language (TCL) commands such as `COMMIT` or `ROLLBACK`.

3.2 Stored Procedures

A named PL/pgSQL block invoked via the `CALL` statement. Procedures do not evaluate to a direct return value and are capable of autonomous transaction control. This capability dictates their use for bulk data manipulation, schema migrations, and batch processing workflows.

4 Parameter Modes

Formal parameters in PL/pgSQL are assigned a mode that dictates the direction of data binding between the calling environment and the subprogram.

- `IN` **Mode (Implicit Default):** Specifies that the formal parameter accepts an actual argument from the caller. It is lexically immutable (read-only) within the subprogram body.
- `OUT` **Mode:** Specifies an uninitialized variable bound to the caller's environment. The subprogram assigns a value to it, which is transmitted back upon termination.
- `INOUT` **Mode:** A hybrid parameter that accepts an initialized actual argument from the caller, permits state mutation within the subprogram body, and transmits the modified state back to the caller upon termination.

5 Defining the Subprograms

Before invoking them dynamically, subprograms must be compiled into the schema.

```
CREATE OR REPLACE FUNCTION calculate_customer_lifetime_value(p_cust_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total_spent NUMERIC;
BEGIN
    SELECT COALESCE(SUM(total_value), 0) INTO v_total_spent
    FROM orders
    WHERE customer_id = p_cust_id;

    RETURN v_total_spent;
END;
$$ LANGUAGE plpgsql;
```

Listing 2: Scalar Function Example

```
CREATE OR REPLACE PROCEDURE process_category_discount(
    p_category IN VARCHAR,      -- Read-only constraint
    p_discount_budget INOUT NUMERIC,   -- Mutated and returned to caller
    p_budget_status OUT VARCHAR -- Assigned and returned to caller
)
LANGUAGE plpgsql AS $$
DECLARE
    v_product_count INT;
BEGIN
    -- Utilize IN parameter for scalar subquery
    SELECT COUNT(*) INTO v_product_count
    FROM products
    WHERE category = p_category;

    -- Transform the INOUT parameter
    p_discount_budget := p_discount_budget - (v_product_count * 50.00);

    -- Initialize the OUT parameter
    IF p_discount_budget > 0 THEN
        p_budget_status := 'BUDGET_SURPLUS';
    ELSE
        p_budget_status := 'BUDGET_DEFICIT';
    END IF;

    -- Procedural Transaction Control
    COMMIT;
END;
$$;
```

Listing 3: Procedure Example with Mixed Modes and TCL

6 Execution Inside Anonymous Blocks

This is the standard methodology for testing or dynamically invoking procedural logic without saving a new script to the database schema. Because anonymous blocks cannot return result sets directly to the client interface like a standard `SELECT` statement, you

must utilize the `DECLARE` section to capture outputs and `RAISE NOTICE` to observe the results.

6.1 Calling a Function

Functions evaluate to a value. Inside a block, you must either assign this evaluated value to a declared variable using the `:=` operator, or use `SELECT ... INTO`.

```
DO $$
DECLARE
    -- 1. Declare a variable to hold the function's return payload
    v_ltv NUMERIC;
BEGIN
    -- 2. Assign the function evaluation directly to the variable
    v_ltv := calculate_customer_lifetime_value(1);

    -- 3. Output the state
    RAISE NOTICE 'Customer LTV: %', v_ltv;
END;
$$;
```

Listing 4: Calling a Function

6.2 Calling a Procedure

Procedures are invoked using the `CALL` statement. When a procedure utilizes `OUT` or `INOUT` parameters, you cannot pass raw literals into those parameter slots. You must pass **variables by reference** so the procedure has a memory address to write the modified state back to.

```
DO $$
DECLARE
    -- 1. Initialize variables for INOUT parameters
    v_promo_budget NUMERIC := 10000.00;

    -- 2. Declare uninitialized variables for OUT parameters
    v_status VARCHAR;
BEGIN
    -- 3. Execute the procedure, passing the variables by reference
    CALL process_category_discount(
        'Electronics',   -- IN argument (literal is fine here)
        v_promo_budget,  -- INOUT argument (must be a variable)
        v_status         -- OUT argument (must be a variable)
    );

    -- 4. The variables in this block's scope now hold the mutated states
    RAISE NOTICE 'Remaining Budget: %, Status: %', v_promo_budget, v_status;
END;
$$;
```

Listing 5: Calling a Procedure

7 Lab Tasks

1. Write a **Stored Procedure** named `evaluate_loyalty_tiers` that accepts an `INOUT` parameter `p_upgraded_count` (initialized to 0). The procedure must loop through all customers, calculate their lifetime spending (sum of `total_value` for `'PROCESSED'` orders), and upgrade their `loyalty_tier` based on the following thresholds: $> \$10,000$ to `'Platinum'`, $> \$5,000$ to `'Gold'`, $> \$1,000$ to `'Silver'`. Do not downgrade anyone. If a customer's tier is changed, increment the `INOUT` counter.

2. Write a **Stored Procedure** named `dynamic_pricing_adjustment` with two `OUT` parameters: `p_increased_count` and `p_decreased_count`. The procedure must iterate through all products. Calculate the total quantity of each product ever sold. If the total quantity sold is strictly greater than 150, increase the product's `base_price` by 10%. If the total sold is strictly less than 120 (including 0), decrease the `base_price` by 15%. Track the number of products modified in both categories.

3. Design a **Stored Procedure** `anonymize_inactive_customers(OUT p_anonymized_count)`. If a customer has never placed an order or their last order was over 730 days ago, execute an anonymization process: change their `name` to `'ANONYMIZED'`, `region` to `'UNKNOWN'`, and `loyalty_tier` to `'Bronze'`. Count updated rows.